MVC Project: Multilayer Perceptron
Artificial Neural Networks — Tasks 1–7

Student Name and Roll No: 25I-2633

National University of Computer and Emerging Sciences (FAST-NUCES)
25I-2633@nu.edu.pk

Abstract. This report presents a complete manual and programmatic implementation of a Multilayer Perceptron (MLP) from first principles. Starting from a raw forward pass on a three-sample student-performance dataset, we derive every computation by hand: weighted sums, sigmoid activations, MSE loss, backpropagation via the chain rule, gradient descent weight updates, mini-batch training, and two optimizers (SGD with Momentum and Nesterov Accelerated Gradient). Finally, the same architecture is scaled to MNIST (60,000 images, 784 inputs, 10 output classes) using NumPy, achieving measurable accuracy after 20 epochs.

Keywords: Multilayer Perceptron · Backpropagation · Gradient Descent · MNIST · Sigmoid Activation · Mini-Batch Training

1 Network Architecture and Assigned Weights

The network used throughout Tasks 1–6 has the structure shown in Table 1.

Table 1. Network Architecture

Layer	Type	Neurons	Activation
Layer 0	Input layer	2	—
Layer 1	Hidden layer 1	2	Sigmoid σ
Layer 2	Hidden layer 2	2	Sigmoid σ
Layer 3	Output layer	1	Sigmoid σ

The network has 10 weights (w_1–w_{10}) and two shared bias terms ($b^{(1)}$ for hidden layer 1, $b^{(2)}$ for hidden layer 2). The output layer has **no bias**. Assigned values for Roll No 25I-2633 (read from the provided Excel file) are given in Table 2.

Table 2. Assigned Weights and Biases — Roll No 25I-2633

w_1	w_2	w_3	w_4	w_5	w_6	w_7	w_8	w_9	w_{10}	$b^{(1)}$	$b^{(2)}$
−0.52	−0.63	0.39	0.56	−0.32	0.63	0.50	−0.21	0.15	−0.23	0.01	−0.11

The dataset used for Tasks 1–6 is a three-sample student performance set (Table 3).

Table 3. Student Performance Dataset

Sample	x_1 (Study Hours)	x_2 (Attendance)	y (Pass?)
$s^{(1)}$	0.2	0.8	1
$s^{(2)}$	0.9	0.4	1
$s^{(3)}$	0.1	0.2	0

2 Task 1 — Forward Pass

2.1 General Equations (Task 1A)

The sigmoid activation and its derivative are:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big) = a(1-a). \tag{1}$$

For each hidden layer l and neuron j:

$$z_j^{(l)} = \sum_i w_{ji}^{(l)}\, a_i^{(l-1)} + b^{(l)}, \qquad a_j^{(l)} = \sigma\Big(z_j^{(l)}\Big). \tag{2}$$

The output neuron has *no bias*:

$$z_1^{(3)} = w_9\, a_1^{(2)} + w_{10}\, a_2^{(2)}, \qquad \hat{y} = \sigma\Big(z_1^{(3)}\Big). \tag{3}$$

2.2 Numerical Forward Pass for $s^{(1)}$ (Task 1B)

Input: $x_1 = 0.2,\ x_2 = 0.8,\ y = 1.$
Hidden Layer 1:

$$z_1^{(1)} = (-0.52)(0.2) + (0.39)(0.8) + 0.01 = -0.1040 + 0.3120 + 0.0100 = 0.2180$$
$$a_1^{(1)} = \sigma(0.2180) = 0.5543$$

$$z_2^{(1)} = (-0.63)(0.2) + (0.56)(0.8) + 0.01 = -0.1260 + 0.4480 + 0.0100 = 0.3320$$
$$a_2^{(1)} = \sigma(0.3320) = 0.5822$$

Hidden Layer 2:

$$z_1^{(2)} = (-0.32)(0.5543) + (0.50)(0.5822) + (-0.11) = -0.1774 + 0.2911 - 0.1100 = 0.0038$$
$$a_1^{(2)} = \sigma(0.0038) = 0.5009$$

$$z_2^{(2)} = (0.63)(0.5543) + (-0.21)(0.5822) + (-0.11) = 0.3492 - 0.1223 - 0.1100 = 0.1169$$
$$a_2^{(2)} = \sigma(0.1169) = 0.5292$$

Output Layer:

$$z_1^{(3)} = (0.15)(0.5009) + (-0.23)(0.5292) = 0.0751 - 0.1217 = -0.0466$$
$$\hat{y} = \sigma(-0.0466) = \mathbf{0.4884}$$

Error: $y - \hat{y} = 1 - 0.4884 = 0.5116$.

2.3 Forward Pass for All Samples (Task 1C)

Table 4. Forward Pass Results for All Three Samples

Sample	x_1	x_2	$a_1^{(1)}$	$a_2^{(1)}$	$a_1^{(2)}$	$a_2^{(2)}$	y	$\hat{y}$
$s^{(1)}$	0.2	0.8	0.5543	0.5822	0.5009	0.5292	1	0.4884
$s^{(2)}$	0.9	0.4	0.4251	0.4175	0.4907	0.5175	1	0.4886
$s^{(3)}$	0.1	0.2	0.5090	0.5147	0.4961	0.5256	0	0.4884

Q1. Sigmoid introduces non-linearity; without it the entire network collapses to a single linear transformation regardless of depth.
Q2. All predictions are ≈ 0.49, effectively random guessing. This shows that randomly initialised weights produce no useful predictions before training.
Q3. Sigmoid output in $(0, 1)$ acts as a probability estimate, making it directly interpretable for binary classification.

3 Task 2 — Loss Calculation

3.1 MSE Formula (Task 2A)

$$L_{\text{MSE}} = \frac{1}{m}\sum_{i=1}^{m}(y^{(i)} - \hat{y}^{(i)})^2 = \frac{1}{3}\Big[(1-\hat{y}^{(1)})^2 + (1-\hat{y}^{(2)})^2 + (0-\hat{y}^{(3)})^2\Big] \quad (4)$$

3.2 Numerical Calculation (Task 2B)

$$e_1 = (1 - 0.4884)^2 = (0.5116)^2 = 0.2618$$
$$e_2 = (1 - 0.4886)^2 = (0.5114)^2 = 0.2615$$
$$e_3 = (0 - 0.4884)^2 = (0.4884)^2 = 0.2385$$
$$\text{SSE} = 0.2618 + 0.2615 + 0.2385 = 0.7618$$
$$L_{\text{MSE}} = 0.7618/3 = \mathbf{0.2539}$$

Q1. A loss of zero would mean perfect predictions for every sample; practically unachievable due to noise and network capacity limits.

Q2. Our loss 0.2539 sits in the middle of $[0, 1]$, confirming the network is random-guessing before training.
Q3. BCE punishes a confidently wrong prediction much more severely than MSE because $\log(\hat{y}) \to -\infty$ as $\hat{y} \to 0$, whereas MSE is capped at 1.

4 Task 3 — Backpropagation

4.1 Delta Definitions

$$\delta_1^{(3)} = -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \tag{5}$$

$$\delta_j^{(l)} = \Big(\sum_k w_{kj}^{(l+1)} \delta_k^{(l+1)}\Big) \cdot a_j^{(l)}(1 - a_j^{(l)}) \tag{6}$$

$$\frac{\partial L}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} \cdot a_i^{(l-1)} \tag{7}$$

4.2 Numerical Backpropagation for $s^{(1)}$ (Task 3B)

Step 1 — Output delta:

$\delta_1^{(3)} = -2(1 - 0.4884) \cdot 0.4884 \cdot (1 - 0.4884) = -2 \times 0.5116 \times 0.2499 = \mathbf{-0.2557}$

Step 2 — Gradients for w_9, w_{10}:

$$\partial L/\partial w_9 = -0.2557 \times 0.5009 = -0.1281$$
$$\partial L/\partial w_{10} = -0.2557 \times 0.5292 = -0.1353$$

Step 3 — Hidden Layer 2 deltas:

$$\delta_1^{(2)} = (0.15 \times -0.2557) \times 0.5009 \times 0.4991 = -0.0096$$
$$\delta_2^{(2)} = (-0.23 \times -0.2557) \times 0.5292 \times 0.4708 = \quad 0.0147$$

Step 4 — Gradients for w_5–w_8, $b^{(2)}$:

$\partial L/\partial w_5 = -0.0053, \quad \partial L/\partial w_6 = 0.0081, \quad \partial L/\partial w_7 = -0.0056, \quad \partial L/\partial w_8 = 0.0085, \quad \partial L/\partial b^{(2)} = 0.0051$

Step 5 — Hidden Layer 1 deltas:

$$\delta_1^{(1)} = [(-0.32)(-0.0096) + (0.50)(0.0147)] \times 0.5543 \times 0.4457 = 0.0026$$
$$\delta_2^{(1)} = [(0.63)(-0.0096) + (-0.21)(0.0147)] \times 0.5822 \times 0.4178 = -0.0022$$

Step 6 — Gradients for w_1–w_4, $b^{(1)}$:

$\partial L/\partial w_1 = 0.0005, \quad \partial L/\partial w_2 = -0.0004, \quad \partial L/\partial w_3 = 0.0021, \quad \partial L/\partial w_4 = -0.0018, \quad \partial L/\partial b^{(1)} = 0.0004$

4.3 Gradient Summary (Task 3C)

Table 5. All Gradients Computed via Backpropagation

Param	Value	Gradient	Param	Value	Gradient
w_1	-0.52	$+0.0005$	w_6	0.63	$+0.0081$
w_2	-0.63	-0.0004	w_7	0.50	-0.0056
w_3	0.39	$+0.0021$	w_8	-0.21	$+0.0085$
w_4	0.56	-0.0018	w_9	0.15	-0.1281
w_5	-0.32	-0.0053	w_{10}	-0.23	-0.1353
$b^{(1)}$	0.01	$+0.0004$	$b^{(2)}$	-0.11	$+0.0051$

Q1. Weights with negative gradients $(w_2, w_4, w_5, w_7, w_9, w_{10})$ must increase; those with positive gradients must decrease.
Q2. When activations are near 0 or 1, $\sigma'(z) = a(1-a) \approx 0$, causing the *vanishing gradient problem*: layers far from the output receive negligible updates and learn very slowly.
Q3. With 12 parameters each in $[-0.9, 0.9]$, brute force search over even 100 values per weight requires $100^{12} = 10^{24}$ evaluations—computationally infeasible. Backpropagation computes the exact gradient direction in a single backward pass.

5 Task 4 — Weight Update and Multiple Iterations

5.1 One Update Step (Task 4A), $\eta = 0.1$

Using $w_{\text{new}} = w_{\text{old}} - 0.1\,\partial L/\partial w$:

Table 6. Updated Weights after One Gradient Descent Step

Param	Old	Gradient	New
w_1	-0.5200	$+0.0005$	-0.5201
w_2	-0.6300	-0.0004	-0.6300
w_3	0.3900	$+0.0021$	0.3898
w_4	0.5600	-0.0018	0.5602
w_5	-0.3200	-0.0053	-0.3195
w_6	0.6300	$+0.0081$	0.6292
w_7	0.5000	-0.0056	0.5006
w_8	-0.2100	$+0.0085$	-0.2109
w_9	0.1500	-0.1281	0.1628
w_{10}	-0.2300	-0.1353	-0.2165
$b^{(1)}$	0.0100	$+0.0004$	0.0100
$b^{(2)}$	-0.1100	$+0.0051$	-0.1105

5.2 Verify Loss Decreased (Task 4B)

Table 7. Loss Before and After One Update

Stage	MSE Loss
Before update (random weights)	0.2539
After 1 update	**0.2527**

The loss decreased by 0.0012, confirming gradient descent is working.
Q1. Loss decreased consistently at $\eta = 0.1$, indicating an appropriate learning rate.
Q2. Training stops when the loss improvement per iteration falls below a small threshold (e.g. $< 10^{-4}$) or after a fixed number of epochs validated against a held-out set.
Q3. Single-sample SGD produces noisy gradient estimates; the update direction can be erratic, causing slow or unstable convergence.

6 Task 5 — Gradient Descent Variants

6.1 Mini-Batch Setup (Task 5A)

With $m = 3$ and $B = 2$: $\lceil 3/2 \rceil = \mathbf{2}$ batches per epoch. Batch 1: $\{s^{(1)}, s^{(2)}\}$; Batch 2: $\{s^{(3)}\}$.
Q1. The mini-batch loss curve is smoother than pure SGD and converges faster per epoch than full-batch GD.
Q2. Shuffling prevents the network from memorising the sample order and ensures each batch is representative of the full dataset.
Q3. For $m = 60{,}000$ and $B = 32$: $\lceil 60000/32 \rceil = 1{,}875$ updates per epoch, versus 1 for batch GD—a 1875× improvement.
Q4. On flat regions or saddle points the gradient is near zero, so plain GD stalls. Momentum carries the update forward using accumulated velocity, escaping these regions.

7 Task 6 — Optimizers

7.1 Momentum Update (Task 6A)

Using $\beta = 0.9$, $\eta = 0.1$, $v_0 = 0$ (scaled variant): $v_t = \beta v_{t-1} + (1 - \beta)\, g_t$, then $w \leftarrow w - \eta v_t$.

On the first step $v_1 = 0.1\, g$, so results are identical to plain GD (Table 6) scaled by 0.1.

7.2 NAG Update (Task 6B)

Because $v_0 = 0$, the lookahead position equals the current position, so Step 1 of NAG gives the same numerical result as Momentum. From Step 2 onward, NAG computes the gradient at the *projected* position $\tilde{w} = w - \eta\beta v_{t-1}$, yielding a more accurate correction and typically faster convergence.
Q1. Momentum and NAG both outperform plain GD after several epochs by accumulating speed in consistent gradient directions.
Q2. NAG converges faster than Momentum theoretically because it "looks ahead" before committing to the step, reducing overshooting.
Q3. Adam is preferred over Momentum/NAG when parameters have vastly different update frequencies (e.g. sparse word embeddings), as it adapts the learning rate per parameter.
Q4. Scaling to MNIST introduces three challenges: (1) much larger weight matrices ($784 \to 128 \to 64 \to 10$) require GPU acceleration; (2) 60,000 samples make per-epoch computation expensive; (3) deeper networks risk vanishing gradients through multiple sigmoid layers.

8 Task 7 — Python Implementation on MNIST

8.1 Overview

The implementation translates every hand-computed step directly into NumPy. The architecture is scaled to $784 \to 128 \to 64 \to 10$ neurons with sigmoid activations, MSE loss, and mini-batch gradient descent ($B = 32$, $\eta = 0.1$, 20 epochs).

8.2 Core Functions

Listing 1.1. Sigmoid activation and its derivative

```
def sigmoid(z):
    z = np.clip(z, -500, 500)      # prevent overflow
    return 1.0 / (1.0 + np.exp(-z))

def sigmoid_derivative(a):
    return a * (1.0 - a)           # same as Task 3
        derivation
```

Listing 1.2. Forward pass — matrix form of Task 1B

```
def forward_pass(X, W1, b1, W2, b2, W3, b3):
    Z1 = X @ W1 + b1;  A1 = sigmoid(Z1)
    Z2 = A1 @ W2 + b2; A2 = sigmoid(Z2)
    Z3 = A2 @ W3 + b3; A3 = sigmoid(Z3)
    return Z1, A1, Z2, A2, Z3, A3
```

Listing 1.3. Backpropagation — matrix form of Task 3B

```
def backpropagation(X, Y_true, Z1,A1,Z2,A2,Z3,A3, W1,W2,W3):
    m = X.shape[0]
    delta3 = -2.0*(Y_true - A3)*sigmoid_derivative(A3)
    dW3 = (A2.T @ delta3)/m
    delta2 = (delta3 @ W3.T)*sigmoid_derivative(A2)
    dW2 = (A1.T @ delta2)/m
    delta1 = (delta2 @ W2.T)*sigmoid_derivative(A1)
    dW1 = (X.T @ delta1)/m
    ...
    return dW1, db1, dW2, db2, dW3, db3
```

Listing 1.4. Mini-batch training loop — Task 5 scaled to MNIST

```
for epoch in range(epochs):
    idx = np.random.permutation(X_train.shape[0])
    for start in range(0, X_train.shape[0], batch_size):
        X_b = X_train[idx[start:start+batch_size]]
        Y_b = Y_train_oh[idx[start:start+batch_size]]
        Z1,A1,Z2,A2,Z3,A3 = forward_pass(X_b, ...)
        grads = backpropagation(X_b, Y_b, ...)
        W1,b1,...= update_weights(..., learning_rate)
```

8.3 Required Outputs

1. **Loss curve** — MSE loss vs. epoch number, visibly decreasing from the initial random-weight baseline to a lower value after 20 epochs, confirming the network is learning.
2. **Test accuracy** — percentage of correctly classified digits on the 10,000-image MNIST test set, computed as accuracy $= \sum[\arg\max(\hat{y}) = y]/m$.
3. **Sample predictions** — a 2×5 grid showing one test image per digit class (0–9) with true and predicted labels; green title = correct, red = incorrect.

8.4 GitHub Repository

The implementation is hosted at:

`https://github.com/25I-2633/mvc-mlp-25I-2633`

Repository structure:

```
mvc-mlp-25I-2633/
  src/     --> MVC_Task7_25I-2633.ipynb
  data/    --> mnist.npz
  report/  --> this PDF report
  README.md
```

9 Conclusion

This project built a Multilayer Perceptron entirely from first principles. Every step — forward propagation, MSE loss, chain-rule backpropagation, gradient descent, mini-batch training, and momentum optimisation — was first derived symbolically, then computed numerically by hand, and finally implemented in NumPy and applied to MNIST. The exercise demonstrates that every matrix multiplication inside a modern neural network library is simply a vectorised version of the scalar arithmetic performed in Tasks 1–6.